E3.16
ISE3.23

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2010

EEE/ISE PART III/IV: MEng, BEng and ACGI

ARTIFICIAL INTELLIGENCE

Corrected Copy

Tuesday, 4 May 10:00 am

Time allowed: 3:00 hours

There are SIX questions on this paper.

Answer FOUR questions.

All questions carry equal marks

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : J.V. Pitt
Second Marker(s) : Y.K. Demiris

© Imperial College London

The Questions

1 a) Explain the difference between *uninformed* and *informed* search, using uniform cost and best first search algorithms to illustrate your answer.

[4]

b) Stew Dent wants to plan a sequence of actions to solve a problem. He has to perform five actions, *a1*, *a2*, *a3*, *a4* and *a5*, once each. Actions can be performed in any order but unfortunately, for a plan to be acceptable there are constraints that have to satisfied: specifically that some actions are performed before others, e.g.:

before(a3, a1).
before(a2, a5).

Thus the plan [*a1, a2, a3, a4, a5*] is not acceptable (for it violates the first constraint), but the plan [*a3, a2, a1, a4, a5*] is acceptable.

Represent, in Prolog or other declarative notation, this planning problem, such that it could be used with the General Graph Search (GGS) algorithm.

[8]

c) In the context of A* search, define a heuristic, explain what it means for a heuristic to be admissible, and explain why admissibility is important in the A* search algorithm.

[4]

d) In A* search, given two arbitrary admissible heuristics, h1 and h2, which composite heuristic is better to use, *max(h1, h2)*, *(h1 + h2)/2*, or *min(h1, h2)*? Justify your answer.

[4]

2 a) There is a basic search problem, consisting of a set of states S, a start state $s0$, a goal test function gt on S such that $gt(s)$ returns true if s is a goal state and false otherwise, and a function $exp(s)$ which returns the set of states obtained by expanding s.

Describe in detail the operation of the General Graph Search (GGS) algorithm for solving a problem of this type.

[10]

b) A graph G can be (explicitly) defined by a 2-tuple $G = \langle N, R \rangle$, where N is a set of nodes, and R is the incidence relation. A rooted graph has a unique node $s \in N$ from which all paths originate.

(i) Give an alternative (inductive) definition of a rooted graph. Show that the paths in the alternative definition are the same as the original (explicit) definition, clearly stating any assumptions you make.

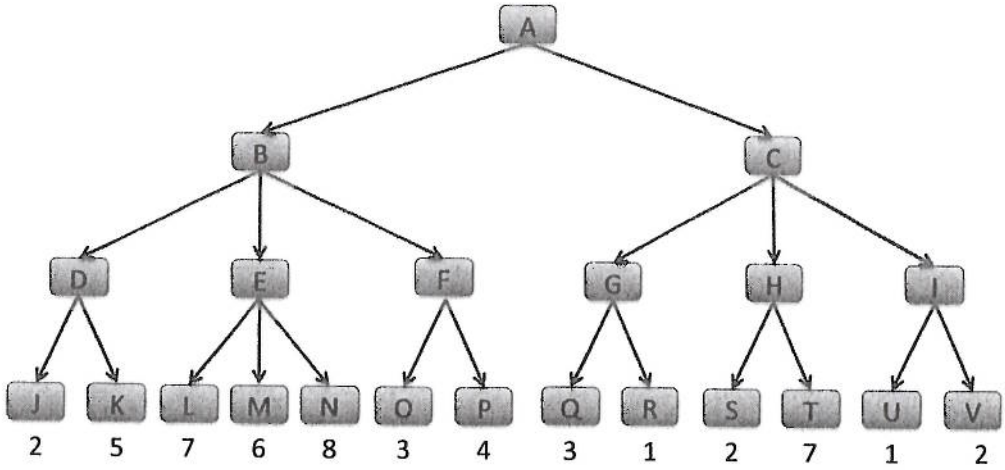
(ii) Explain how the General Graph Search (GGS) algorithm explores the paths induced by the alternative definition, using breadth first and depth first search strategies.

[10]

3 a) Describe the Minimax algorithm for two-player games; firstly, when exhaustive search is possible, and secondly, when it is not. Explain why the Alphabeta algorithm is a better way to search game trees.

[6]

b) Consider the following game tree, and assume that the first player to move is MAX.



(i) Using the Minimax algorithm, show which move should MAX make;

(ii) Using the Alphabeta algorithm, show which branches would be pruned, and show which move MAX should make.

[8]

c) (i) Both the Minimax and Alphabeta algorithms depend on certain assumptions about the game and the players. State those assumptions.

(ii) Most game-playing programs do not save search results from one move to the next. Instead, they usually start completely afresh whenever it is the machine's turn to move. Suggest reasons why.

(iii) Explain why search in game-playing programs always proceeds forwards from the current state rather than backward from the goal state(s).

[6]

4 a) Describe a procedure for converting a set of first-order formulas into clausal form, i.e. where every conjunct is of the form:

$$\neg a1 \vee \neg a2 \vee \ldots \vee \neg am \vee b1 \vee b2 \vee \ldots \vee bn$$

[4]

b) The villagers have found a witch (or so they think) and want to burn her. Consider the following statements of Sir Bedevere.

For anything, if it is a woman and it burns, then it is a witch.
For anything, if it is made of wood, then it burns.
For anything, if it floats, then it is made of wood.
For any two things, if the first thing floats, and it weighs the same as the second thing, then the second thing floats.

Express these four statements as formulas of First Order Predicate Logic, and translate them into clausal form.

[6]

c) Consider the following facts:

By observation, *the girl is a woman.*
By observation, *the duck floats.*
By experiment, *the duck weighs the same as the girl.*

Express these facts in clausal form. Explain why existential quantifiers have not been used.

[4]

d) Prove, using resolution and showing the unifiers, that *the girl is a witch* [and so the villagers should, in fact, burn her (Burn her! Burn her! etc.)].

[6]

5 a) Given an inference system I, and letting S be a set of formulas and p a proposition, explain what is meant by the following statements:

(i) $S \models p \rightarrow S \vdash_I p$

(ii) $S \vdash_I p \rightarrow S \models p$

Is it better for I to be sound, but not complete; or unsound, but complete? Justify your answer.

[4]

b) Explain the difference between conjunctive normal form and disjunctive normal form.

Explain the relationship between disjunctive normal form and *KE* tableaux.

[4]

c) Using the *KE* proof procedure, decide whether each of the following sentences of propositional logic is valid, satisfiable, or invalid.

(i) $smoke \rightarrow smoke$

(ii) $(smoke \rightarrow fire) \rightarrow (\neg smoke \rightarrow \neg fire)$

(iii) $(smoke \rightarrow fire) \rightarrow ((smoke \wedge alarm) \rightarrow fire)$

[6]

d) Suppose that a software agent has the following sentences in its knowledge base *KB*:

$KB = \{p, (q \wedge w), (p \wedge q) \rightarrow (s \vee z), (a \vee r \vee \neg z), \neg a\}$

Using the *KE* proof procedure, show $KB \models (s \vee r)$.

[6]

6 a) Describe how the syntax of propositional logic can be extended to give modal logic.

[2]

b) Define a *Kripke model*, and specify how a Kripke model is used to give a semantics for formulas of modal logic.

[4]

c) Consider the following statements (Aristotle's Sea Battle):

If I give the order to attack, then necessarily there will be a battle.
If I do not give the order to attack, then necessarily there will not be a battle.
Now, either I give the order, or I do not give the order.
Therefore, necessarily there will be a battle or necessarily there will not be a battle..

This can be formulated in modal logic either as:

1.
$$\begin{array}{l} p \rightarrow \Box q \\ \neg p \rightarrow \Box \neg q \\ \hline p \vee \neg p \\ \hline \Box q \vee \Box \neg q \end{array}$$

2.
$$\begin{array}{l} \Box(p \rightarrow q) \\ \Box(\neg p \rightarrow \neg q) \\ \hline p \vee \neg p \\ \hline \Box q \vee \Box \neg q \end{array}$$

Using the KE tableau method for modal logic **K**, show that for one of these formulations, the argument is valid; and for the other, not.

Give a Kripke model that shows the argument is not valid (the model should make all three premises true, but the conclusion false).

[8]

d) Show, using Kripke models, or otherwise, that the following formulas are not valid in modal logic **K**:

(i) $(\Diamond p \wedge \Diamond q) \rightarrow \Diamond(p \wedge q)$

(ii) $(p \rightarrow \Box q) \leftrightarrow \Box(p \rightarrow q)$

(iii) $\Box p \rightarrow \Diamond p$

[6]

Solutions - A.I. 2010 E3.16

[ste 3.23]

1 (a) bookwork
uninformed search based on what we know about the search space we have seen
informed search based on what we know about the problem
uniform cost, expand node on search frontier with least cumulative path cost from the start node as computed by cost function given by g
best first, expand node on search frontier with least estimated path cost to a (the) goal node a computed by heuristic function given by h

[4]

(b) [application]

```
state_change( _, Alist, [A | Alist] ) :-
        action( A ),
        \+ member( A, Alist).

goal_state( Alist ) :-
        length( Alist, 5 ),
        test_each_A( Alist ).

test_each_A( [A|As] ) :-
        no_befores( A, As ),
        test_each_A( As ).

no_befores( A, [] ).

no_befores( Later, [ Earlier | Rest ] ) :-
        \+ before( Later, Earlier ),
        no_befores( Later, Rest ).
```

node representation (state, g-cost, h-cost)
state: the actual representation of the problem
g-cost: actual path cost from start state to node
h-cost: estimated path cost from node to goal state

[8]

(c) [bookwork/understanding]
heuristic is a function/algorithm which produces an approximate solution to a problem. Here the problem is to compute the path cost from a node to a goal state. A heuristic computes an approximate cost.

Admissible heuristic never over-estimates the actual cost.

If over-estimate then we might pick a path that led to a sub-optimal goal if the heuristic kept on thinking that nodes on this path were better than the estimate for the node on the path to the optimal goal. Therefore important in proving optimality.

[4]

(d)[understanding]

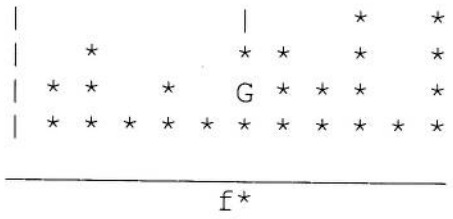
h_composite = max(h1, h2)

taking h_composite as above is admissible because both h1 and h2 are admissible.

Then we maximise h-value without ever over-estimating. This means we are likely to expand fewer nodes.

To see why:

Let f* be cost of optimal node.
A* expands all nodes with f-cost less than f*.
A* expands some nodes with f-cost = f*.
A* expands no nodes with f-cost > f*.
Since f(n) = g(n) + h(n), this means that A* expands all those nodes such that h(n) < f* - g(n).
In other words, the more nodes for which this relation holds, the more nodes will be expanded by A* using this heuristic, and the less efficiently will the search space be explored.
Alternatively, consider histogram of nodes according to actual f-cost, whereby f-actual(n) = g(n) + h-actual(n).



Only those nodes to the left of the f* bar will be expanded. In practice of course, we don't have h-actual we just have h. Thus we could have a redistribution of the histogram which pushes more of the nodes to the left of the f* bar.
In other words, we want to ensure f* < f(n) + h(n) < f-actual(n)

[4]

Grand total [20]

2

(a) [bookwork/understanding]

state is a representation of problem
node is a collection of information, including state
path is a sequence of nodes
graph is a set of paths

a graph G contains a solution path, if

pick a path P in the graph, and
s = state of frontier node N of P, and
gt(s) = true

or

pick a path P in the graph, and
P' = G – {P}, and
s = state of frontier node N of P, and
S = exp(s), and
P'' = P ++ <make_node(s',…)>, for each s' in S, and
G' = P' union P'', and
G' contains a solution path

Key points are definition of pick and implementation of union, which gives different algorithms.

[10]

(b) [bookwork/application]

(i)

Alternative definition

G' = < start_node, Op > where

start_node is the root node
op is set of state transformers op: node -> node

Assume op computes all the members of incidence relation

Then the inductive definition of the paths graph is given by:

$P'_G = \bigcup_{i=0}^{\infty} P'_i$

where

$P'_0 = \{ (<s>) \}$

$P'_{i+1} = \{ (p_i ++ <n_{i+1}>, (g+e+h(n)) \mid \exists op \in Op . \exists (p_i,g) \in P'_i . (n_{i+1},e) = op(frontier(p_i)) \}$

The explicit definition is given by:

$Paths(G) = \bigcup_{i=0}^{\infty} Pi$

Where

$P0 = \{ <s> \}$

$Pi+1 = \{(p ++ <n>, (g+e+h(n)) \mid \exists(p,g) \in Pi.\ frontier(p) = (n') \ \&\ (n',e,n) \in R \}$

Since P'0 = P0, then every P'i+1 = Pi+1, on

(ii)
Depth first picks member of P'0. Then P'1. Then P'2, until P'm, where m is the maximum depth of the tree, and nowhere left to go. Then other elements of P'm. If nowhere to go, try other members of P'(m-1), and so on.

Breadth first tries P'0. Then every member of P'1. Then every member of P'2. And so on until P'd, where d is the depth of the solution

Total [10]

Grand Total [20]

3
(a) [bookwork/application]
max is player trying to win, or MAXimize advantage
min is opponent who attempts to MINimize max's score.
First case: exhaustively generate the graph
each leaf node is given a score of 1 or 0, depending on whether the state is a win for max or min respectively
propagate leaf values up the graph according to the rules:
if the parent is a MAX, give it the minimum value of its children
if the parent is a MIN, give it the maximum value of its children

Second case (exhaustive search not possible): generate graph to ply (depth)
Each leaf node given a heuristic score
propagate leaf values up the graph according to the rules as before

alphabeta stores alpha value with max node which can never decrease on a path
stores beta value with min mode which can never increase on a path

search depth first
offer alpha and beta values as cutoff values for pruning search

Search stopped below any MIN node having a beta value
less than or equal to alpha value of any of its MAX ancestors
Search stopped below any MAX node having an alpha value
greater than or equal to beta value of any of its MIN ancestors

To see why:
Suppose at MAX node with alpha value N.
Suppose branch search returns value M, M > N
There is no point searching the other branches
If another branch returns a value less than M, then it will 'lose' to M in computing the MAX value for this node.
If another branch returns a value greater than M, then it will 'lose' to N in computing the MIN value for an ancestor.

Similarly:
Suppose at MIN node with beta value N.
Suppose branch search returns value M, M < N
There is no point searching the other branches
If another branch returns a value greater than M, then it will 'lose' to M in computing the Min value for this node.
If another branch returns a value less than M, then it will 'lose' to N in computing the MAX value for an ancestor.

If the ordering of the nodes is favourable, then alphabeta can effectively double the depth of search.

[6]

(b) [application]
DEF = 584, GHI = 372

BC = 42
A= 4
So Minimax returns 4 and so choose B

B offers E 5 as alpha cutoff
L = 7 and > 5, so no further search (M=6 loses to 7 at this MAX propagation, N=8 would lose to 5 at the MIN propagation even if the 4 was not returned by F at the MIN node)
O = 3 and < 5, so carry on the search
B returns 4

A offers C 4 as beta cutoff
G = 3 and < 4, so no further search (H=7 loses to 3 at this MIN propagation, V=2 will lose to 4 at the MAX propagation)

So Alphabeta prunes M under E and N under E

… and prunes H under C and I under C

Alphabeta returns 4 (alphabeta returns same results as Minimax) and so choose B

[8]

(c) [understanding]
(i)
Assume that min uses the same information as max and attempts always to move to a state that is worst for max.
Assume perfect information.
Assume same information

(ii) The values computed by the heuristic function at the new ply may give rise to different values being propagated up the tree, therefore the branches pruned by alphabeta may be different, etc

(iii) exhaustive search not possible
too many goal states

[6]

Grand Total [20]

4

(a) [bookwork]

eliminate implication and equivalence
reduce scope of negation
rename variables
move quantifiers left without changing order
skolemize
drop universal quantifiers
convert to conjunctive normal form
make each conjunct a separate clause
give variables with same name in different clauses, different names

Total [4]

(b) [application]

$\forall x \,.\, woman(x) \wedge burns(x) \rightarrow witch(x)$
$\forall x \,.\, wood(x) \rightarrow burns(x)$
$\forall x \,.\, floats(x) \rightarrow wood(x)$
$\forall x, y \,.\, floats(x) \wedge weighssame(x, y) \rightarrow floats(y)$

$\neg woman(X) \vee \neg burns(X) \vee witch(X)$
$\neg wood(X) \vee burns(X)$
$\neg floats(X) \vee wood(X)$
$\neg floats(X) \vee \neg weighssame(X, Y) \vee floats(Y)$

[6]

(c) [*understanding/application*]

woman(girl)
floats(duck)
weighssame(duck, girl)

not use existential quantifier because we want it to be the same duck

[4]

(d) [*application*]

prove witch(girl)

start with negated conclusion (proof by refutation)

$\neg witch(\ girl\ )$
$\neg woman(girl) \vee \neg burns(girl)\ \{X = girl\}$
$\neg burns(girl)$
$\neg wood(girl)\ \{X = girl\}$
$\neg floats(girl)\ \{X = girl\}$
$\neg floats(X) \vee \neg weighssame(X, girl)\ \{Y = girl\}$
$\neg floats(duck)\ \{X = duck\}$
nil

[6]
Grand Total [20]

And of course we have as an axiom
$\forall x \,.\, witch(x) \rightarrow burn(villagers, x)$
☺

5

(a) [bookwork/understanding]

First off, $\models$ means S entails p: any valuation (assignment of truth values to propositional symbols) which makes every formula in S true, also makes p true

Then, $\vdash_I$ means S proves p in inference system I: starting from S, there is a sequence of formulas, where each formula follow from the previous one(s) by a rule of inference of KE, and ending in p (strictly speaking, we prove |-I (S -> p) and we use refutation, but the |- relation is the same)

Then

(i) is completeness: if there is an entailment, then I can prove it, i.e. $\models$ implies $\vdash_I$

(ii) is soundness: if I proves something, then it is an entailment, i.e. $\vdash_I$ implies $\models$

Better to be sound but not complete (hello Prolog). Any deep discussions of the value of unsound inference gets credit though.

Total [4]

(b) [bookwork]

conjunctive normal form is conjunction of disjunctions
disjuntive normal form is a disjunction of conjunctions

- *KE proof procedure*
- *to prove Q, begin with ¬Q and search for a refutation*
- *expand ¬Q according to, but clearing away, logical structure of Q*
- *expansion takes form of tree, called a tableau, with formulas labelling nodes*
- *tableau is representation of disjunctive normal form*
- *each branch represents the conjunction of formulas on the branch*
- *the tableau is a disjunction of its branches*

Total [4]

c) [application]

(i)

¬conc	¬(*smoke* → *smoke*)	
1	*smoke*	*alpha*, *p*1
2	¬*smoke*	*alpha*, *p*1

close

= *valid*

(*ii*)

¬conc	¬[(*smoke* → *fire*) → (¬*smoke* → ¬*fire*)]	
1	*smoke* → *fire*	*alpha*, *p*1
2	¬(¬*smoke* → ¬*fire*)	*alpha*, *p*1
3	¬*smoke*	*alpha*, 2
4	¬¬*fire*	*alpha*, 2
5	*fire*	¬¬ *elim*, 4

(1 *is beta*¬*simplified*)

open

so let smoke = false and fire = true

(*false* → *true*) → (*true* → *false*) == *true* → *false* == *false*

so it is not valid, is it satisfiable? So try it the other way:

-c $(smoke \rightarrow fire) \rightarrow (\neg smoke \rightarrow \neg fire)$			
1	$(smoke \rightarrow fire)$		*pb*1
2	$(\neg smoke \rightarrow \neg fire)$		*beta*, *p*1, 1
	3	$smoke$	*pb*1
	4	$fire$	*beta* 1, 3
	5	$\neg\neg smoke$	*beta* 2, 4
	6	$smoke$	$\neg\neg$ *elim* 5
	open		

So smoke = true, fire = true would do

$(true \rightarrow true) \rightarrow (false \rightarrow false) == true \rightarrow true == true$

So it is satisfiable

(iii)

-c	$\neg[(smoke \rightarrow fire) \rightarrow ((smoke \wedge alarm) \rightarrow fire)]$	
1	$(smoke \rightarrow fire)$	*alpha*, *p*1
2	$\neg((smoke \wedge alarm) \rightarrow fire)$	*alpha*, *p*1
3	$smoke \wedge alarm$	*alpha*, 2
4	$\neg fire$	*alpha*, 2
5	$smoke$	*alpha*, 3
6	$alarm$	*alpha*, 3
7	$fire$	*beta*, 1, 5
close		

So valid (monotonicity of implication)

Total [6]

d) [application]

premises	$p, (q \wedge w), (p \wedge q) \rightarrow (s \vee z), (a \vee r \vee \neg z), \neg a$		
$\neg$*conc*	$\neg(s \vee r)$		
1	$\neg s$		*alpha*, $\neg$*conc*
2	$\neg r$		*ditto*
3	q		*alpha*, *prem*
4	w		*alpha*, *prem*
5	$r \vee \neg z$		*beta*, *prem prem*
6	$\neg z$		*beta*, 2, 5
	7	$\neg(p \wedge q)$	*pb*1*a*
		8 $\neg p$	*pb*2*a*
		close (*prem*, 8)	
		9 p	*pb*2*b*
		10 $\neg q$	*beta*, 7, 9
		close (3,10)	
	11	$(p \wedge q)$	*pb*1*a*
	12	$(s \vee z)$	*beta*, *prem*, 8
	13	z	*beta*, 1, 12
		close (6, 13)	

Total [6]

Grand Total [20]

6
(a) [bookwork]
wff ::= □ ***wff*** | ◊ ***wff***

Total [2]

Kripke model M
M = <W, R, ||>
Where W is non-empty set of worlds
R is accessibility relation on W
|| is denotation function which maps propositions onto subsets of W

Meaning of modal formulas

$\models_{M,a} \Box p$ *is true* $\leftrightarrow \forall w \,.\, aRw \rightarrow \models_{M,w} p$
$\models_{M,a} \Diamond p$ *is true* $\leftrightarrow \exists w \,.\, aRw \wedge \models_{M,w} p$

[4]

(c) [application]

1: $p \rightarrow \Box q$	*prem*
1: $\neg p \rightarrow \Box \neg q$	*prem*
1: $p \vee \neg p$	*prem*
1: $\neg(\Box q \vee \Box \neg q)$	*¬conc*
1: $\neg \Box q$	*alpha*
1: $\neg \Box \neg q$	*alpha*
1,1: $\neg q$	◊ *rule*
1,2: q	◊ *rule* [*note different world label unrestricted*]
1: p	*pb1a*
1: $\Box q$	*beta*
1,1: q	□ *rule*
close	
1: $\neg p$	*pb1b*
1: $\Box \neg q$	*beta*
1,2: $\neg q$	□ *rule*
close	

(ii)

1: $\Box(p \rightarrow q)$	*prem*
1: $\Box(\neg p \rightarrow \neg q)$	*prem*
1: $p \vee \neg p$	*prem*
1: $\neg(\Box q \vee \Box \neg q)$	*¬conc*
1: $\neg \Box q$	*alpha*
1: $\neg \Box \neg q$	*alpha*
1,1: $\neg q$	◊ *rule*
1,2: q	◊ *rule* [*note different world label unrestricted*]
1,1: $p \rightarrow q$	
1,1: $\neg p \rightarrow \neg q$	
1,1: $\neg p$	
1,2: $p \rightarrow q$	
1,2: $\neg p \rightarrow \neg q$	
1,2: p	

So
$W = \{ [1], [1,1], [1,2] \}$
$R = \{ ([1], [1,1]), ([1], [1,2]) \}$
$||p|| = \{[1], [1,2]\}$
$||q|| = \{[1,2]\}$
makes all three premises true, and the conclusion false

[8]

(d) [application]
All not valid
(i)
W = { [1], [1,1], [1,2] }
R = { ([1], [1,1]), ([1], [1,2]) }
||p|| = {[1,1]}
||q|| = {[1,2]}
would do

(ii)
W = { [1], [1,1] }
R = { ([1], [1]), ([1], [1,1]) }
||p|| = {[1]}
||q|| = {[1]}
would do

(ii)
W = { [1]}
R = { }
||p|| = {[1]}
would do

[6]
Grand Total [20]